

Lesson 2 Teacher's Key

Unity

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Requires using UnityEngine.InputSystem; at the top of the script (added alongside using UnityEngine;), and a fresh Unity project's default Active Input Handling of "Both" - see the Handout's Setup/Trouble notes. Attach to the Player GameObject:

```
void Update() {
    float h = 0f;
    if (Keyboard.current.rightArrowKey.isPressed) h = 1f;
    if (Keyboard.current.leftArrowKey.isPressed) h = -1f;
    transform.Translate(new Vector3(h, 0, 0) * 3f * Time.deltaTime);
}
```

Level 2 - Complete Worked Solution

Extend the Level 1 script to use both axes:

```
public float speed = 3f;

void Update() {
    float h = 0f;
    if (Keyboard.current.rightArrowKey.isPressed) h = 1f;
    if (Keyboard.current.leftArrowKey.isPressed) h = -1f;
    float v = 0f;
    if (Keyboard.current.upArrowKey.isPressed) v = 1f;
    if (Keyboard.current.downArrowKey.isPressed) v = -1f;
    transform.Translate(new Vector3(h, v, 0) * speed * Time.deltaTime);
}
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

The simplest blink in Unity is toggling SpriteRenderer's built-in enabled property (inherited from Renderer) on and off based on an accumulating timer - no Animator or sprite swap needed:

```
private SpriteRenderer sr;
private float blinkTime = 0f;
void Start() { sr = GetComponent<SpriteRenderer>(); }
```

```
void Update() {
    float h = 0f;
    if (Keyboard.current.rightArrowKey.isPressed) h = 1f;
    if (Keyboard.current.leftArrowKey.isPressed) h = -1f;
    float v = 0f;
    if (Keyboard.current.upArrowKey.isPressed) v = 1f;
    if (Keyboard.current.downArrowKey.isPressed) v = -1f;
    bool moving = h != 0 || v != 0;
    if (moving) {
        blinkTime += Time.deltaTime;
        sr.enabled = ((int)(blinkTime * 6) % 2 == 0);
    } else {
        sr.enabled = true;
        blinkTime = 0f;
    }
}
```

Sound: beep-beep while moving, silent when idle

Add an AudioSource component and assign sfx_blip.wav as its AudioClip. Checking .isPlaying before calling .Play() again means it naturally retriggers as soon as the short clip finishes - a beep-beep rhythm, not one continuous tone. Calling .Stop() when idle is what makes it actually silent - PlayOneShot() alone has no matching "stop early" call:

```
public AudioSource audioSrc;

if (moving) {
    if (!audioSrc.isPlaying) audioSrc.Play();
} else {
    audioSrc.Stop();
}
```

Recolor: different sprite per direction, reset when idle

Same Idle/Moving concept as the State Diagram from this lesson's bonus section. This swaps the Sprite itself rather than tinting it with sr.color - add 5 public Sprite fields (spriteDefault, spriteRight, spriteLeft, spriteUp, spriteDown) and drag the 5 placeholder images onto them in the Inspector:

```
    if (h == 0 && v == 0) sr.sprite = spriteDefault;   // idle
    else if (h > 0) sr.sprite = spriteRight;
    else if (h < 0) sr.sprite = spriteLeft;
    else if (v > 0) sr.sprite = spriteUp;
    else sr.sprite = spriteDown;
```

Clamp movement to the screen

The -8/8 and -4.5/4.5 bounds approximate the visible camera area for Unity's default 2D orthographic camera at size 5 - adjust to match whatever camera size the class ends up using:

```
    Vector3 pos = transform.position;
    pos.x = Mathf.Clamp(pos.x, -8f, 8f);
    pos.y = Mathf.Clamp(pos.y, -4.5f, 4.5f);
    transform.position = pos;
```

Structure: group everything triggered by the same action in one place

Unity's Update() already runs once per frame, so the "same place in code" for this lesson is that one method - the trick is organizing what goes inside it, not spreading logic across multiple methods or components. Movement and the screen clamp always run, every frame, regardless of direction. Blink, sound, and recolor are all direction-dependent, so they all live under one shared if (moving) / else check - not four separate direction checks scattered through the method. That's the Complete Level 3 solution below.

Complete Level 3 solution (all 5 goals combined)

Everything above combined into one script, attached to the Player GameObject (with an AudioSource component, AudioClip set to sfx_blip.wav):

```
public float speed = 3f;
public AudioSource audioSrc;
public Sprite spriteDefault, spriteRight, spriteLeft, spriteUp,
spriteDown;
private SpriteRenderer sr;
private float blinkTime = 0f;

void Start() {
    sr = GetComponent<SpriteRenderer>();
}

void Update() {
    float h = 0f;
    if (Keyboard.current.rightArrowKey.isPressed) h = 1f;
    if (Keyboard.current.leftArrowKey.isPressed) h = -1f;
    float v = 0f;
    if (Keyboard.current.upArrowKey.isPressed) v = 1f;
    if (Keyboard.current.downArrowKey.isPressed) v = -1f;
    transform.Translate(new Vector3(h, v, 0) * speed * Time.deltaTime);

    Vector3 pos = transform.position;
    pos.x = Mathf.Clamp(pos.x, -8f, 8f);
    pos.y = Mathf.Clamp(pos.y, -4.5f, 4.5f);
    transform.position = pos;

    bool moving = h != 0 || v != 0;
    if (moving) {
        blinkTime += Time.deltaTime;
        sr.enabled = ((int)(blinkTime * 6) % 2 == 0);
        if (!audioSrc.isPlaying) audioSrc.Play();
        if (h > 0) sr.sprite = spriteRight;
        else if (h < 0) sr.sprite = spriteLeft;
        else if (v > 0) sr.sprite = spriteUp;
        else sr.sprite = spriteDown;
    } else {
        sr.enabled = true;
        blinkTime = 0f;
        audioSrc.Stop();
        sr.sprite = spriteDefault;
    }
}
```

Alternate Path Notes: Visual Scripting

Unity's Visual Scripting graphs compile to the same underlying calls as the C# path - no performance or capability loss for a beginner project.

Watch for students forgetting to wire the flow (white/execution) connection from the On Input System Event Vector 2 node's Trigger output into Translate - a common visual-scripting mistake that looks identical to the graph "doing nothing."

If the Input System nodes don't show up in the fuzzy finder at all, the Visual Scripting Node Library needs to be regenerated once per project (Project Settings > Visual Scripting > Regenerate Nodes) - easy to forget since it's a one-time setup step, not something students do every lesson.

If Move doesn't appear in the On Input System Event Vector 2 node's Input Action dropdown, the Player Input component's Actions field is probably still empty - it needs the PlayerControls asset assigned before Visual Scripting can see any of its actions.

The Level 3 goals map onto nodes the same way the code maps onto members: enabled toggles to a "Set Member" node on Sprite Renderer, .isPlaying/.Play()/.Stop() to Audio Source nodes, and sprite to another "Set Member" node on Sprite Renderer (with the 5 placeholder images wired in as Sprite variables) - all wired under one Branch node keyed off the same moving check used for movement.

- Visual Scripting is part of Unity's own Junior Programmer pathway content, so this stays aligned with the certification track for students who prefer it.
- The node-graph structure (event -> data -> action) is the same conceptual shape as the C# script - worth pointing out explicitly.
- Using the On Input System Event Vector 2 node (bound to a Move Input Action) instead of the old Get Axis node keeps the Visual Scripting path genuinely aligned with the C# path's new Input System, rather than secretly still depending on the legacy Input Manager underneath a different-looking node.